

Afeez Bolaji Usman

Student ID: 202514326WY

Course Code: SBT-DF202

Course Title : Computer and Digital Forensics

Lab Title: Lab 4 Steganography and Hidden Data Detection

Instructor Name: Aminu Idris

Date: 4/9/2026

Executive Summary

This practical investigated the use of steganography to conceal information within an image file and demonstrated techniques for embedding, extracting, detecting, and verifying hidden data.

A BMP image was used as the carrier file. A controlled evidence note named secret.txt was created and embedded into a copy of the carrier image using Steghide with the password 1234. The hidden file was subsequently extracted successfully and its SHA-256 hash was compared with the original file.

StegSeek was then used to perform a controlled dictionary attack against the stego image. The correct password, 1234, was successfully identified because it was included in the controlled wordlist.

An independent steganography exercise was also completed. A separate note was created, hashed, embedded into a new BMP image using the password Lab4@2026, extracted successfully, and verified using SHA-256. StegSeek was again able to identify the password when it was included in the test wordlist.

The practical demonstrated that steganography can conceal information without obvious changes to the normal file format, while weak passwords protecting steganographic content may be vulnerable to dictionary-based attacks.

Lab Objectives

The objectives of this practical were to:

- Understand the difference between steganography and encryption.
- Understand basic data-hiding techniques, including LSB-based hiding.
- Prepare and preserve a carrier image.
- Create and hash a controlled evidence note.
- Embed hidden information into an image using Steghide.
- Extract the hidden information and verify its integrity.
- Compare the original and stego images.
- Perform a controlled password dictionary test using StegSeek.
- Complete an independent hidden-note exercise.
- Document the investigation in a reproducible manner.

Tools and Resources Used

- Kali Linux — forensic analysis environment
- Steghide — embedding and extracting hidden data
- StegSeek — password dictionary testing
- SHA-256 (sha256sum) — evidence integrity verification
- file — file type identification
- xxd — hexadecimal inspection
- strings — readable-text analysis
- BMP carrier image — _tower_original_image_for_lab.bmp
- Controlled wordlists — password testing

Methodology

The investigation was carried out in a controlled Kali Linux environment. The original BMP image was preserved and its SHA-256 hash was calculated. A controlled evidence note was then created, hashed, and embedded into a copy of the image using Steghide. The hidden file was extracted and its hash compared with the original to verify integrity.

The original and stego images were compared using file size, SHA-256, hexadecimal inspection, and strings. StegSeek was then used with controlled wordlists to test password recovery. Finally, an independent hidden note was created, embedded, extracted, and verified using the same forensic procedures.

Part A — Prepare and Preserve the Evidence

Download a bumb picture

```

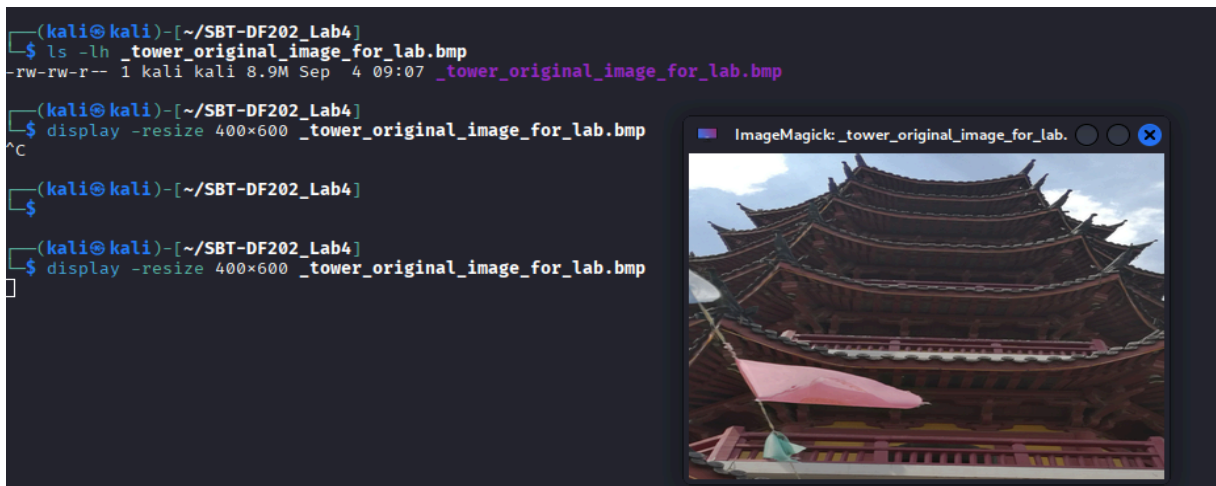
(kali@kali)-[~]
└─$ mkdir -p ~/SBT-DF202_Lab4/{evidence,hashes,working,extracted,screenshots,output}
cd ~/SBT-DF202_Lab4

(kali@kali)-[~/SBT-DF202_Lab4]
└─$ ls -la
total 32
drwxrwxr-x 8 kali kali 4096 Sep  4 09:06 .
drwx----- 39 kali kali 4096 Sep  4 09:06 ..
drwxrwxr-x 2 kali kali 4096 Sep  4 09:06 evidence
drwxrwxr-x 2 kali kali 4096 Sep  4 09:06 extracted
drwxrwxr-x 2 kali kali 4096 Sep  4 09:06 hashes
drwxrwxr-x 2 kali kali 4096 Sep  4 09:06 output
drwxrwxr-x 2 kali kali 4096 Sep  4 09:06 screenshots
drwxrwxr-x 2 kali kali 4096 Sep  4 09:06 working

(kali@kali)-[~/SBT-DF202_Lab4]
└─$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/steganography/_tower_original_image_for_lab.bmp

(kali@kali)-[~/SBT-DF202_Lab4]
└─$ ls -lh _tower_original_image_for_lab.bmp
-rw-rw-r-- 1 kali kali 8.9M Sep  4 09:07 _tower_original_image_for_lab.bmp

```



```

(kali@kali)-[~/SBT-DF202_Lab4]
└─$ file _tower_original_image_for_lab.bmp
_tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cb
Size 9277494, bits offset 54

(kali@kali)-[~/SBT-DF202_Lab4]
└─$ sha256sum _tower_original_image_for_lab.bmp | tee hashes/tower_original_image.sha256
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db _tower_original_image_for_lab.bmp

(kali@kali)-[~/SBT-DF202_Lab4]
└─$

```

The authorised laboratory carrier image `_tower_original_image_for_lab.bmp` was downloaded and examined. The file utility confirmed that it is a Windows BMP image with a resolution of 1365 × 2265 pixels and 24-bit color depth.

A SHA-256 hash was calculated before performing any steganography operations:

```
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db
```

The original carrier image was preserved and will not be overwritten. A separate stego image will be created for subsequent embedding activities.

Part B — Create the Secret Evidence File

```
(kali㉿kali)-[~/SBT-DF202_Lab4]
$ nano secret.txt

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cat secret.txt
Full Name: [Afeez Bolaji]
Evidence Message: This is a controlled evidence note created for the steganography practical lab.
Purpose: This file was created for the IC DFA Lab 4 – Steganography and Hidden Data Detection.

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ sha256sum secret.txt | tee hashes/secret_original.sha256
0f0b3538be33fdaf9a851099cd9fc8af6b2c33aa060e3ea637a61abd0578adb secret.txt

(kali㉿kali)-[~/SBT-DF202_Lab4]
```

Part C — Embed the Hidden File with Steghide

```
(kali㉿kali)-[~/SBT-DF202_Lab4]
$ pwd
/home/kali/SBT-DF202_Lab4
$ ls -lh _tower_original_image_for_lab.bmp secret.txt
-rw-rw-r-- 1 kali kali 220 Sep  4 09:26 secret.txt
-rw-rw-r-- 1 kali kali 8.9M Sep  4 09:07 _tower_original_image_for_lab.bmp

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ steghide embed \
  -ef secret.txt \
  -cf _tower_original_image_for_lab.bmp \
  -sf tower_stego_lab4.bmp \
  -p 1234
embedding "secret.txt" in "_tower_original_image_for_lab.bmp" ... done
writing stego file "tower_stego_lab4.bmp" ... done

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ ls -lh tower_stego_lab4.bmp
-rw-rw-r-- 1 kali kali 8.9M Sep  4 10:03 tower_stego_lab4.bmp

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ file tower_stego_lab4.bmp
tower_stego_lab4.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494,
bits offset 54

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ sha256sum tower_stego_lab4.bmp | tee hashes/tower_stego_lab4.sha256
625cddf3c105b4d758598bc0db1deb76ec17367b464fa56ac2cca7862f6aef42 tower_stego_lab4.bmp
```

Steghide was used to embed the controlled evidence file secret.txt into a new BMP image named tower_stego_lab4.bmp. The laboratory password 1234 was used as required.

The original carrier image was preserved without modification. The resulting stego image retained the same BMP format, dimensions, and color depth as the original carrier.

The SHA-256 hash of the resulting stego image was:

625cddf3c105b4d758598bc0db1deb76ec17367b464fa56ac2cca7862f6aef42

The successful creation of the stego image confirmed that the secret evidence file was embedded into the carrier image.

Part D — Extract and Verify the Hidden Evidence

```
(kali㉿kali)-[~/SBT-DF202_Lab4]
$ mkdir -p extracted

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ steghide extract \
-sf tower_stego_lab4.bmp \
-xf extracted/extracted_secret.txt \
-p 1234
wrote extracted data to "extracted/extracted_secret.txt".

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cat extracted/extracted_secret.txt
Full Name: [Afeez Bolaji]
Evidence Message: This is a controlled evidence note created for the steganography practical lab.
Purpose: This file was created for the ICDFA Lab 4 – Steganography and Hidden Data Detection.

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ sha256sum secret.txt extracted/extracted_secret.txt
0f0b3538be33fdfaf9a851099cd9fc8af6b2c33aa060e3ea637a61abd0578adb  secret.txt
0f0b3538be33fdfaf9a851099cd9fc8af6b2c33aa060e3ea637a61abd0578adb  extracted/extracted_secret.txt
```

The concealed secret.txt file was successfully extracted from tower_stego_lab4.bmp using Steghide and the correct password (1234).

The extracted file was examined and contained the expected controlled evidence message. SHA-256 hashes were then calculated for both the original and extracted files.

Both files produced the same SHA-256 hash:

0f0b3538be33fdfaf9a851099cd9fc8af6b2c33aa060e3ea637a61abd0578adb

This confirms that the extracted evidence file is bit-for-bit identical to the original secret.txt and demonstrates successful hidden-data extraction and integrity verification.

Part E — Compare Original and Stego Images

```
(kali㉿kali) [~/SBT-DF202_Lab4]
$ ls -lh tower_original_image_for_lab.bmp tower_stego_lab4.bmp
-rw-r--r-- 1 kali kali 8.9M Sep 4 09:07 tower_original_image_for_lab.bmp
-rw-r--r-- 1 kali kali 8.9M Sep 4 10:03 tower_stego_lab4.bmp

(kali㉿kali) [~/SBT-DF202_Lab4]
$ sha256sum tower_original_image_for_lab.bmp tower_stego_lab4.bmp
6508bebb94bd8ec0833bf85580cda4261d89632ddc89ef7f9f7093b5e653cd4db tower_original_image_for_lab.bmp
625cddf3c105b4d758598bc0ddb1deb76ec17367b464fa56ac2cca7862f6aeaf42 tower_stego_lab4.bmp

(kali㉿kali) [~/SBT-DF202_Lab4]
$ xxd -l 128 tower_original_image_for_lab.bmp
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800    BM6 .....6 ... (.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000    ..U.....
00000020: 0000 0000 0000 120b 0000 120b 0000 0000    ....
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f    .... "5 #8. 8.
00000040: 203a 2425 3f28 2945 2729 482a 2b4d 2a2c    :$(?)E')H**+M*.
00000050: 4e29 2e4f 282e 5127 3052 2831 5328 3254    N).O(.Q'OR(1S(2T
00000060: 2f39 5b35 4163 3d4e f644 5979 465f 8147    /9[SAC=NoDyYF_G
00000070: 6687 4669 8b45 6889 4468 8e45 6785 4a69    f.F.i.Eh.Dh.Eg.Ji

(kali㉿kali) [~/SBT-DF202_Lab4]
$ xxd -l 128 tower_stego_lab4.bmp
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800    BM6 .....6 ... (.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000    ..U.....
00000020: 0000 0000 0000 120b 0000 120b 0000 0000    ....
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f    .... "5 #8. 8.
00000040: 203a 2425 3f28 2945 2729 482a 2b4d 2a2c    :$(?)E')H**+M*.
00000050: 4e29 2e4f 282e 5127 3052 2831 5328 3254    N).O(.Q'OR(1S(2T
00000060: 2f39 5b35 4163 3d4e f644 5979 465f 8147    /9[SAC=NoDyYF_G
00000070: 6687 4669 8b45 6889 4468 8e45 6785 4a69    f.F.i.Eh.Dh.Eg.Ji

(kali㉿kali) [~/SBT-DF202_Lab4]
$ strings tower_original_image_for_lab.bmp | head -20
"5 #8
:$(?)E')H**+M*).N).O(.Q'OR(1S(2T/9[SAC=NoDyYF_
Gf.F.i.Eh.Dh.Eg.Ji
?#7?adfiotwx|ryuyvzdeixYJMNRONR]|fe\[_GLHLIMNOSRSWOOUIOEEKEKQPQUQRVQTWXZ^chdgldi`afRRXMMSSSV[YaRUy\cmptilp`cg
[X_VY]hbfcig_cafiknktokhlqaejoxsfJOBFk8C7-B@AE9BfBAE9Bf-FJ-dK7?FA8E;CGCEI:C69BF8AE9BF;DH:BT9AH9AH9AH;CJ<DK9AH8B
GT9AH8BI7A7HA7HB7IA7HA7FM;DK:DKBBI;EL:DKBBI7A7HAHO>FM;CJ;CJ?GN?BN:BTA9AH>CG<DK<DK<EL>FM=EL<DL?HN:IMBC9GDHT7JN?
JXF:FJDKO=IM&kKgG-GK9D9;FJWJ?JNA&O-HL>Im&kkoJUUYZeifauo-zq|
kvxrlp2NALCeRmQMCPtA7ALNFQCCTEPENRALPKOBQBaQLCCPRALEP
xyofwfwdsfm&KPERISGXGPRFX5FWGJAOUSZFPMRGOGYGOGIXFLVCRHWMDTRLELOKFYLBELFTFKULSZTSZTSZTSZTFWPCWLUCILMIUMBYHVPYHPVHFVGZYOHVPH
```

The original carrier image and the resulting stego image were compared using file size, SHA-256 hashing, hexadecimal inspection, and readable-string analysis.

Both images remained approximately 8.9 MB and retained the same BMP format, dimensions, and 24-bit color depth. However, their SHA-256 hashes were different:

Original carrier: 6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db

Stego image: **625cddf3c105b4d758598bc0db1deb76ec17367b464fa56ac2cca7862f6aef42**

The first 128 bytes examined with xxd were identical between the two files, indicating that the initial BMP header and beginning of the image data were preserved.

The strings examination did not reveal the hidden evidence message as readable plaintext. Instead, the output consisted primarily of binary/pixel-data characters. This is expected because steganographic data is concealed within the image data rather than simply appended as visible text.

The comparison demonstrates that the stego image maintains the normal appearance and structure of the BMP file while containing modifications required to store the hidden evidence.

Part F — Controlled Password Dictionary Test

```
(kali@kali)-[~/SBT-DF202_Lab4]
$ cd ~/SBT-DF202_Lab4
printf "password\nadmin\n1234\nletmein\n" > working/wordlist.txt

(kali@kali)-[~/SBT-DF202_Lab4]
$ cat working/wordlist.txt
password
admin
1234
letmein

(kali@kali)-[~/SBT-DF202_Lab4]
$ cd ~/SBT-DF202_Lab4
stegseek tower_stego_lab4.bmp working/wordlist.txt extracted/stegseek_recovered.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1234"
[i] Original filename: "secret.txt".
[i] Extracting to "extracted/stegseek_recovered.txt".

(kali@kali)-[~/SBT-DF202_Lab4]
$ cd ~/SBT-DF202_Lab4
cat extracted/stegseek_recovered.txt
Full Name: [Afeez Bolaji]
Evidence Message: This is a controlled evidence note created for the steganography practical lab.
Purpose: This file was created for the ICDA Lab 4 – Steganography and Hidden Data Detection.

(kali@kali)-[~/SBT-DF202_Lab4]
$ sha256sum secret.txt extracted/stegseek_recovered.txt
0f0b3538be33fdfaf9a851099cd9fc8af6b2c33aa060e3ea637a61abd0578adb secret.txt
0f0b3538be33fdfaf9a851099cd9fc8af6b2c33aa060e3ea637a61abd0578adb extracted/stegseek_recovered.txt
```

A controlled password dictionary containing four candidate passwords was created:

password
admin
1234
letmein

StegSeek 0.6 was then used against the stego image tower_stego_lab4.bmp with the controlled wordlist.

StegSeek successfully identified the embedded passphrase:

1234

The tool also identified the original embedded filename as secret.txt and extracted the hidden data to:

extracted/stegseek_recovered.txt

The recovered file was examined and contained the expected controlled evidence message.

SHA-256 verification produced the following results:

Original secret.txt: 0f0b3538be33fdfaf9a851099cd9fc8af6b2c33aa060e3ea637a61abd0578adb

StegSeek recovered file: 0f0b3538be33fdfaf9a851099cd9fc8af6b2c33aa060e3ea637a61abd0578adb

The identical SHA-256 hashes confirm that the file recovered through the StegSeek password dictionary test is bit-for-bit identical to the original hidden evidence file.

This demonstrates that a weak, guessable steganographic password can be successfully recovered through a dictionary-based password attack when the correct password is included in the candidate wordlist.

Part G — Independent Student Task

```
(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cd ~/SBT-DF202_Lab4

cat > working/independent_note.txt <<'EOF'
Independent Steganography Test
This is my independent hidden note for ICDFA Lab 4.
The purpose is to demonstrate embedding, hashing, and password testing.
EOF

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cat working/independent_note.txt
Independent Steganography Test
This is my independent hidden note for ICDFA Lab 4.
The purpose is to demonstrate embedding, hashing, and password testing.

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cd ~/SBT-DF202_Lab4
sha256sum working/independent_note.txt | tee hashes/independent_note.sha256
b92452260339754595ed240f10498912dad908b101f674f9e87d9d3956caf42c  working/independent_note.txt

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cat hashes/independent_note.sha256
b92452260339754595ed240f10498912dad908b101f674f9e87d9d3956caf42c  working/independent_note.txt

(kali㉿kali)-[~/SBT-DF202_Lab4]
$
```

```
(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cd ~/SBT-DF202_Lab4

steghide embed \
-ef working/independent_note.txt \
-cf _tower_original_image_for_lab.bmp \
-sf output/independent_stego.bmp \
-p 'Lab4@2026'
embedding "working/independent_note.txt" in "_tower_original_image_for_lab.bmp" ... done
writing stego file "output/independent_stego.bmp" ... done
```

```
(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cd ~/SBT-DF202_Lab4
ls -lh output/independent_stego.bmp
file output/independent_stego.bmp
sha256sum output/independent_stego.bmp | tee hashes/independent_stego.sha256
-rw-rw-r-- 1 kali kali 8.9M Sep  4 10:48 output/independent_stego.bmp
output/independent_stego.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9
277494, bits offset 54
7148a4931378fe2916d067ae420cdf665fe2ed819479f4aa0d8389a23a3a7dfd  output/independent_stego.bmp
```

```
(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cd ~/SBT-DF202_Lab4

steghide extract \
-sf output/independent_stego.bmp \
-xf extracted/independent_recovered.txt \
-p 'Lab4@2026'
wrote extracted data to "extracted/independent_recovered.txt".

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cat extracted/independent_recovered.txt
Independent Steganography Test
This is my independent hidden note for ICDFA Lab 4.
The purpose is to demonstrate embedding, extraction, hashing, and password testing.

(kali㉿kali)-[~/SBT-DF202_Lab4]
$
```

```
(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cd ~/SBT-DF202_Lab4

sha256sum working/independent_note.txt extracted/independent_recovered.txt
b92452260339754595ed240f10498912dad908b101f674f9e87d9d3956caf42c  working/independent_note.txt
b92452260339754595ed240f10498912dad908b101f674f9e87d9d3956caf42c  extracted/independent_recovered.txt

(kali㉿kali)-[~/SBT-DF202_Lab4]
$
```



```

(kali㉿kali)-[~]
$ cd ~/SBT-DF202_Lab4

printf "password\nadmin\n1234\nLab4@2026\nletmein\n" > working/independent_wordlist.txt

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cat working/independent_wordlist.txt
password
admin
1234
Lab4@2026
letmein

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ stegseek output/independent_stego.bmp \
working/independent_wordlist.txt \
extracted/stegseek_independent_recovered.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "Lab4@2026"
[i] Original filename: "independent_note.txt".
[i] Extracting to "extracted/stegseek_independent_recovered.txt".

(kali㉿kali)-[~/SBT-DF202_Lab4]
$ cd ~/SBT-DF202_Lab4
sha256sum working/independent_note.txt extracted/stegseek_independent_recovered.txt
b92452260339754595ed240f10498912dad908b101f674f9e87d9d3956caf42c  working/independent_note.txt
b92452260339754595ed240f10498912dad908b101f674f9e87d9d3956caf42c  extracted/stegseek_independent_recovered.txt

```

Security and Forensic Observations

The practical produced several important forensic observations.

Steganography versus Encryption

Encryption transforms information into ciphertext so that the content is unreadable without the appropriate key. Steganography instead attempts to conceal the existence of the information by embedding it within another file.

File Hashing

SHA-256 hashing was used to establish integrity before and after extraction. Matching hashes for the original and recovered notes confirmed that the recovered evidence was identical to the source note.

Stego Image Detection

A stego image may retain the same file type, dimensions, and approximate size as the original carrier. Therefore, basic file inspection alone may not reveal that information has been concealed.

Dictionary Attacks

The StegSeek tests demonstrated that weak or predictable passwords can be vulnerable to dictionary attacks. In both controlled exercises, the correct password was recovered because it was deliberately included in the test wordlist.

Evidence Preservation

The original carrier image was preserved and separate output files were created for steganographic operations. This is important in digital forensics because original evidence should not be unnecessarily modified.

Conclusion

The SBT-DF202 Lab 4 practical successfully demonstrated the principles and forensic considerations associated with steganography and hidden data detection.

The controlled evidence note was successfully embedded into a BMP carrier using Steghide, extracted using the correct password, and verified using SHA-256 hashing. Comparison of the original and stego images showed that the files retained the same general BMP characteristics while producing different complete-file hashes.

The StegSeek controlled dictionary attack successfully identified the weak password 1234. The independent exercise further demonstrated the same process using a separately selected password, Lab4@2026.

The matching SHA-256 hashes between original and recovered notes provide evidence that the extraction process preserved the contents of the hidden files.

Overall, the exercise demonstrated that forensic investigators should consider steganography when examining suspicious media and should use file analysis, hashing, specialized steganography tools, and controlled password testing where legally and operationally authorized.